ΠΑΝΕΛΛΑΔΙΚΕΣ ΕΞΕΤΑΣΕΙΣ

ΗΜΕΡΗΣΙΩΝ – ΕΣΠΕΡΙΝΩΝ ΕΠΑΓΓΕΛΜΑΤΙΚΩΝ ΛΥΚΕΙΩΝ

ΤΡΙΤΗ 10 ΙΟΥΝΙΟΥ 2025

ΕΞΕΤΑΖΟΜΕΝΟ ΜΑΘΗΜΑ: ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΥΠΟΛΟΓΙΣΤΩΝ

ΘΕΜΑ Α

Α1. Να χαρακτηρίσετε τις προτάσεις που ακολουθούν, γράφοντας στο τετράδιό σας, δίπλα στο γράμμα που αντιστοιχεί σε κάθε πρόταση, τη λέξη Σωστό, αν η πρόταση είναι σωστή, ή τη λέξη Λάθος, αν η πρόταση είναι λανθασμένη.

α. Η δυαδική αναζήτηση εκμεταλλεύεται τη διάταξη των στοιχείων ενός συνόλου δεδομένων για τη γρήγορη εύρεση ενός στοιχείου.

β. Μία βιβλιοθήκη (library), σε οποιαδήποτε γλώσσα προγραμματισμού, είναι μία συλλογή εργαλείων που μπορεί να έχουν γραφτεί και από άλλους προγραμματιστές, προκειμένου να εκτελούνται συγκεκριμένες λειτουργίες.

γ. Η έκφραση "1000" < "2" επιστρέφει τιμή False.

δ. Η συνάρτηση divmod(x,y) επιστρέφει τη δύναμη του x υψωμένη στο y.

ε. Η αρίθμηση των δεικτών στις λίστες στην Python ξεκινάει από το 0.

Α. ΣΩΣΤΟ, Β. ΣΩΣΤΟ, Γ.ΛΑΘΟΣ, Δ.ΛΑΘΟΣ, Ε.ΣΩΣΤΟ

Α2. Να γράψετε στο τετράδιό σας τους αριθμούς 1, 2, 3, 4, 5 από τη στήλη Α του παρακάτω πίνακα και δίπλα σε καθέναν από τους αριθμούς, ένα από τα γράμματα α, β, γ, δ, ε, στ της στήλης Β, που δίνει τη σωστή αντιστοίχιση. Σημειώνεται ότι ένα γράμμα από τη στήλη Β θα περισσέψει.

ΣΤΗΛΗ Α	**ΣΤΗΛΗ Β**
1. not(34!=45)	**α.** 4
2. (12<11) or (23>10)	**β.** 5
3. 45%10	**γ.** True
4. 4/(2**0)	**δ.** False
5. 45/10.0	**ε.** 4.0
	στ. 4.5

1. δ 2. γ 3. β 4. α 5. στ

ΘΕΜΑ Β

Β1. Δίνεται η παρακάτω κλάση:

class Politis:

  def __init__(self, onoma, poliseis):

    self.onoma=onoma

    self.poliseis=poliseis

όπου onoma είναι το ονοματεπώνυμο του πωλητή και poliseis είναι μία λίστα με τα μηνιαία έσοδα από τις πωλήσεις που πραγματοποιήθηκαν κατά τη διάρκεια τεσσάρων (4) μηνών.

α) Να δημιουργήσετε μέθοδο με όνομα max_poso(), η οποία υπολογίζει και επιστρέφει το μεγαλύτερο μηνιαίο ποσό των πωλήσεών του.

β) Να δημιουργήσετε ένα στιγμιότυπο της κλάσης Politis ως εξής: politis1 με τιμές των ιδιοτήτων του, onoma="Ιωάννα Κωνσταντίνου", poliseis=[10000, 15000, 5000, 20000].

γ) Για το στιγμιότυπο politis1 να βρείτε και να εμφανίσετε το μεγαλύτερο ποσό πωλήσεων με χρήση της μεθόδου max_poso() που περιγράφεται στο ερώτημα α).

```
class Politis:
    def __init__(self, onoma, poliseis):
        self.onoma = onoma
        self.poliseis = poliseis

    # Ερώτημα α)
    def max_poso(self):

        megisto = self.poliseis[0] # Είναι προτιμότερο να αρχικοποιούμε με το πρώτο στοιχείο της λίστας

        # Εναλλακτικά, αντί για 4, το len() κάνει τη μέθοδο να δουλεύει για όσους μήνες κι αν έχουμε
        for i in range(1, len(self.poliseis)):
            if self.poliseis[i] > megisto:
                megisto = self.poliseis[i]

        return megisto

# --- ΚΥΡΙΩΣ ΠΡΟΓΡΑΜΜΑ ---

# Ερώτημα β) Δημιουργία στιγμιοτύπου (αντικειμένου)
politis1 = Politis("Ιωάννα Κωνσταντίνου", [10000, 15000, 5000, 20000])

# Ερώτημα γ) Κλήση μεθόδου και εμφάνιση
m = politis1.max_poso()
print m
```

Μέσα στη μέθοδο max_poso(self), η προσπέλαση στη λίστα των πωλήσεων ***πρέπει οπωσδήποτε*** *να γίνεται με self.poliseis[i] και όχι απλά με poliseis[i]. Το self είναι αυτό που συνδέει τη μέθοδο με τα δεδομένα του συγκεκριμένου αντικειμένου (της κας Ιωάννας στην περίπτωσή μας).*

Β2. Η παρακάτω συνάρτηση δέχεται μία λίστα (listA) που περιέχει λέξεις και επιστρέφει μία συμβολοσειρά που δημιουργείται από τα αρχικά γράμματα των λέξεων της λίστας (listA).

```
def newWord (listA):
  arxika =  (1)
  for  (2)  in listA :
    arxika =  (3) + lexi[  (4) ]
  return  (5)
```

Να γράψετε στο τετράδιό σας τους αριθμούς (1), (2), (3), (4) και (5) που αντιστοιχούν στα κενά της παραπάνω συνάρτησης και δίπλα σε καθέναν από τους αριθμούς αυτό που πρέπει να συμπληρωθεί, ώστε η συνάρτηση να εκτελεί την αναφερόμενη λειτουργία.

1. '' *(ή "")*
2. lexi
3. arxika
4. 0
5. arxika

```
def newWord(listA):
    arxika = ''     # (1) Αρχικοποίηση κενής συμβολοσειράς (accumulator)
    for lexi in listA: # (2) Το lexi διατρέχει μία-μία τις λέξεις της λίστας
        arxika = arxika + lexi[0] # (3),(4) Προσθέτουμε το 1ο γράμμα (θέση 0) στο τέλος της arxika
    return arxika     # (5) Επιστρέφουμε το τελικό αποτέλεσμα
```

Β3. Δίνεται η συμβολοσειρά word="ΠΡΟΤΥΠΟ ΕΠΑΛ". Να γράψετε στο τετράδιό σας τι θα εμφανίσει η εκτέλεση του παρακάτω τμήματος προγράμματος:

```
print "1.",word[:7]
print "2.",word[:]
print "3.",word[3:11]
print "4.",word[8:len(word)]
print "5.",word[0]+word[8]
```

Ανάλυση των δεικτών (indices) της συμβολοσειράς:
Π(0) Ρ(1) Ο(2) Τ(3) Υ(4) Π(5) Ο(6) (7) Ε(8) Π(9) Α(10) Λ(11)

Ανάλυση των Αποτελεσμάτων:

1. word[:7]: Από την αρχή έως τη θέση 6 (το 7 εξαιρείται).
   - Αποτέλεσμα: ΠΡΟΤΥΠΟ
2. word[:]: Ολόκληρη η συμβολοσειρά.
   - Αποτέλεσμα: ΠΡΟΤΥΠΟ ΕΠΑΛ

3. word[3:11]: Από τη θέση 3 έως τη θέση 10.
    - 3 είναι το Τ και 10 είναι το Α.
    - Αποτέλεσμα: ΤΥΠΟ ΕΠΑ
4. word[8:len(word)]: Το len(word) είναι 12. Άρα από τη θέση 8 έως τη θέση 11.
    - Η θέση 8 είναι το Ε, η 11 είναι το Λ.
    - **Διόρθωση**: Το αποτέλεσμα είναι ΕΠΑΛ (Χωρίς το κενό μπροστά). Το κενό βρίσκεται στη θέση 7, ενώ εσύ ξεκινάς από την 8.
5. word[0]+word[8]: Το πρώτο γράμμα (θέση 0) και το ένατο γράμμα (θέση 8).
    - word[0] -> Π
    - word[8] -> Ε
    - Το αποτέλεσμα είναι ΠΕ.

ΘΕΜΑ Γ

Ένα πλανητάριο δέχεται τέσσερα (4) σχολεία ανά ημέρα, για πέντε (5) ημέρες. Θεωρήστε ότι κατά το διάστημα αυτό κάθε σχολείο επισκέπτεται το πλανητάριο μόνο μία φορά. Το πλήθος των μαθητών ανά σχολείο είναι από είκοσι (20) έως και πενήντα (50) μαθητές. Η τιμή του εισιτηρίου για κάθε μαθητή είναι τέσσερα (4) ευρώ.

Να αναπτύξετε πρόγραμμα σε γλώσσα προγραμματισμού Python το οποίο:

Γ1. Για κάθε μία από τις πέντε (5) ημέρες και για κάθε σχολείο να διαβάζει:

α) Το όνομα του σχολείου.

β) Τον τύπο του σχολείου ("ΕΠΑΛ" για Επαγγελματικό Λύκειο, "ΠΕΠΑΛ" για Πρότυπο Επαγγελματικό Λύκειο, "ΓΕΛ" για Γενικό Λύκειο, "ΜΟΥΣ" για Μουσικό σχολείο).

γ) Το πλήθος των μαθητών του σχολείου. Να γίνεται έλεγχος ορθότητας, ώστε το πλήθος των μαθητών που καταχωρίζονται να είναι από είκοσι (20) έως και πενήντα (50) μαθητές.

Γ2. Να υπολογίζει και να εμφανίζει το κόστος των εισιτηρίων για κάθε σχολείο. Για τον σκοπό αυτό να γίνεται κλήση της συνάρτησης YPOLOGISMOS(), όπως περιγράφεται στο ερώτημα Γ3.

Γ3. Να υλοποιεί συνάρτηση με όνομα YPOLOGISMOS, η οποία δέχεται το πλήθος των μαθητών ενός σχολείου και επιστρέφει το κόστος των εισιτηρίων του σχολείου.

Γ4. Να υπολογίζει και να εμφανίζει:

α) Τα συνολικά έσοδα του πλανηταρίου από τις επισκέψεις των σχολείων. (μον.3)

β) Το ποσοστό των μαθητών των Πρότυπων Επαγγελματικών Λυκείων (ΠΕΠΑΛ) που επισκέφθηκαν το πλανητάριο κατά τη διάρκεια των πέντε (5) ημερών.

Σημείωση: Θεωρήστε ότι υπάρχει τουλάχιστον ένα (1) σχολείο από κάθε τύπο σχολείου στη διάρκεια των πέντε (5) ημερών.

```
# Γ3: Ορισμός της συνάρτησης στην αρχή του προγράμματος
def YPOLOGISMOS(m):
    return m * 4.0 # Επιστρέφει το κόστος: πλήθος μαθητών * 4 ευρώ (το .0 εξασφαλίζει
float)

# Γ4.α & Γ4.β: Αρχικοποίηση αθροιστών και μετρητών
sk = 0.0           # Συνολικά έσοδα πλανηταρίου από όλα τα σχολεία
s1 = 0             # Συνολικό πλήθος μαθητών όλων των σχολείων
s2 = 0             # Συνολικό πλήθος μαθητών μόνο από ΠΕΠΑΛ

# Γ1: Διπλή επανάληψη για 5 ημέρες και 4 σχολεία ανά ημέρα
for i in range(5):          # Εξωτερική επανάληψη για τις ημέρες (0 έως 4)
    for j in range(4):      # Εσωτερική επανάληψη για τα σχολεία (0 έως 3)
        # Γ1.α & Γ1.β: Είσοδος δεδομένων κειμένου (raw_input για Python 2.7)
        on = raw_input('Όνομα σχολείου: ')
        t = raw_input('Τύπος σχολείου: ')

        # Γ1.γ: Έλεγχος ορθότητας για το πλήθος των μαθητών (20 έως 50)
        pm = int(input('Πλήθος μαθητών: '))
        while pm < 20 or pm > 50:
            pm = int(input('Λάθος τιμή. Εαναδώστε πλήθος (20-50): '))

        # Γ2: Κλήση της συνάρτησης για τον υπολογισμό κόστους του συγκεκριμένου
σχολείου
        k = YPOLOGISMOS(pm)
        print 'ΚΟΣΤΟΣ ΣΧΟΛΕΙΟΥ:', k

        # Ενημέρωση των συνολικών μεγεθών
        sk = sk + k               # Προσθήκη κόστους στα συνολικά έσοδα
        s1 = s1 + pm              # Προσθήκη μαθητών στο γενικό σύνολο

        # Έλεγχος αν το σχολείο είναι ΠΕΠΑΛ για τον υπολογισμό του ποσοστού αργότερα
        if t == 'ΠΕΠΑΛ':
            s2 = s2 + pm        # Προσθήκη μαθητών ΠΕΠΑΛ στον ειδικό αθροιστή

print 'Συνολικά έσοδα: ', sk # Γ4.α: Εμφάνιση συνολικών εσόδων μετά το τέλος όλων των
επαναλήψεων

pososto = float(s2) / s1 * 100 # Γ4.β: Υπολογισμός ποσοστού με χρήση float για αποφυγή
ακέραιας διαίρεσης

print 'Το ποσοστό μαθητών ΠΕΠΑΛ είναι: ', pososto, '%'
```

Διδακτικές Επισημάνσεις για τους Μαθητές:

1. *Η "Δύναμη" της Εμφωλευμένης Επανάληψης: Για κάθε μία επανάληψη του i (ημέρα), η επανάληψη του j (σχολεία) εκτελείται τέσσερις φορές. Συνολικά, το σώμα της εσωτερικής for θα εκτελεστεί 5 * 4 = 20 φορές.*
2. *Γιατί Συνάρτηση; Ο YPOLOGISMOS κάνει τον κώδικα πιο καθαρό. Αν αύριο η τιμή του εισιτηρίου αλλάξει, θα την αλλάξουμε μόνο σε ένα σημείο (μέσα στη συνάρτηση) και όχι παντού στο πρόγραμμα.*
3. *Αμυντικός Προγραμματισμός: Η while για τον έλεγχο ορθότητας είναι απαραίτητη. Η συνθήκη της while είναι η συνθήκη λάθους (πότε θέλουμε να ξαναρωτήσουμε τον χρήστη;).*
4. *Η παγίδα της διαίρεσης: Στην Python 2.7, η έκφραση s2 / s1 αν και τα δύο είναι ακέραιοι (π.χ. 10 / 40) θα έδινε 0. Η χρήση της float(s2) "αναγκάζει" την Python να κάνει δεκαδική διαίρεση.*

ΘΕΜΑ Δ

Ένας φορέας Πιστοποίησης Γλωσσομάθειας διεξάγει ετήσιες εξετάσεις πιστοποίησης στη γλώσσα των Ισπανικών στις οποίες προσήλθαν τριακόσιοι (300) υποψήφιοι. Οι υποψήφιοι πρέπει να συμπληρώσουν ένα ηλεκτρονικό φύλλο αξιολόγησης που αποτελείται από δέκα (10) ερωτήσεις πολλαπλής επιλογής. Κάθε ερώτηση έχει τέσσερις (4) δυνατές απαντήσεις, οι οποίες χαρακτηρίζονται από τα γράμματα a, b, c και d. Μόνο μία από τις απαντήσεων a, b, c, d είναι η σωστή. Θεωρήστε ότι ο υποψήφιος για κάθε ερώτηση επιλέγει υποχρεωτικά μία και μόνο απάντηση.

Κάθε σωστή απάντηση βαθμολογείται με πέντε (5) μονάδες, ενώ κάθε λανθασμένη βαθμολογείται με μηδέν (0) μονάδες. Οι σωστές απαντήσεις βρίσκονται στη λίστα ANS=["a", "d", "b", "b", "a", "c", "d", "a", "b", "c"].

Να αναπτύξετε πρόγραμμα σε γλώσσα προγραμματισμού Python το οποίο:

Δ1. Για κάθε υποψήφιο:

α) Να διαβάζει τον κωδικό του (ως συμβολοσειρά) και να τον καταχωρίζει σε λίστα με όνομα KOD.

β) Να διαβάζει από το πληκτρολόγιο διαδοχικά τις δέκα (10) απαντήσεις που έδωσε και να υπολογίζει τη συνολική βαθμολογία του, σύμφωνα με την περιγραφόμενη διαδικασία βαθμολόγησης και χρήση της λίστας ANS. Η λίστα ANS πρέπει να οριστεί στο πρόγραμμα.

γ) Να καταχωρίζει τη συνολική βαθμολογία του στη λίστα SB.

Δ2. Να εμφανίζει τους κωδικούς των υποψηφίων που είχαν βαθμολογία μεγαλύτερη των τριάντα (30) μονάδων. Θεωρήστε ότι υπάρχει τουλάχιστον ένας (1) υποψήφιος με βαθμολογία μεγαλύτερη των τριάντα (30) μονάδων.

Δ3. α) Να ταξινομεί τη λίστα SB με χρήση του αλγορίθμου ταξινόμησης της ευθείας ανταλλαγής (φυσαλίδα – bubble sort) σε φθίνουσα σειρά, αναδιατάσσοντας συγχρόνως τη λίστα KOD, ώστε να διατηρείται η αντιστοιχία κωδικών και συνολικών βαθμολογιών.

β) Να δημιουργεί αρχείο κειμένου με όνομα "lang.txt", όπου σε κάθε γραμμή του θα γραφεί ο κωδικός του υποψηφίου, ακολουθούμενος από έναν κενό χαρακτήρα και τη συνολική του βαθμολογία.

Σημείωση: Δεν απαιτούνται έλεγχοι ορθότητας δεδομένων.

```
# Ορισμός της λίστας με τις σωστές απαντήσεις (Δεδομένο εκφώνησης)
ANS = ['a', 'd', 'b', 'b', 'a', 'c', 'd', 'a', 'b', 'c']
SB = []      # Λίστα για την αποθήκευση των τελικών βαθμολογιών
KOD = []     # Λίστα για την αποθήκευση των κωδικών των υποψηφίων

# Δ1: Εισαγωγή δεδομένων και βαθμολόγηση
for i in range(300):                  # Επανάληψη για κάθε έναν από τους 300 υποψήφιους
    k = raw_input('Κωδικός υποψηφίου: ')     # Δ1.α: Διάβασμα κωδικού ως συμβολοσειρά
    KOD.append(k)                            # Καταχώριση κωδικού στη λίστα KOD

    s = 0                        # Αρχικοποίηση βαθμολογίας (sum) για τον τρέχοντα υποψήφιο
    for j in range(10):          # Δ1.β: Εσωτερική επανάληψη για τις 10 ερωτήσεις
        ap = raw_input('Δώσε απάντηση: ')    # Διάβασμα απάντησης (a, b, c ή d)
        if ap == ANS[j]:         # Σύγκριση απάντησης με τη σωστή στη θέση j της λίστας ANS
            s = s + 5                        # Αν είναι σωστή, πρόσθεση 5 μονάδων

    SB.append(s)                 # Δ1.γ: Καταχώριση της συνολικής βαθμολογίας στη λίστα SB

# Δ2: Εμφάνιση κωδικών με βαθμολογία > 30
for i in range(300):
```

```
    if SB[i] > 30:                  # Έλεγχος αν ο υποψήφιος στη θέση i ξεπέρασε το 30
        print KOD[i]                # Εμφάνιση του αντίστοιχου κωδικού

# Δ3.α: Ταξινόμηση Φυσαλίδας (Bubble Sort) σε φθίνουσα σειρά
N = 300
for i in range(N - 1):                  # Εξωτερικός βρόχος περασμάτων
    for j in range(N - 1, i, -1):       # Εσωτερικός βρόχος συγκρίσεων από το τέλος προς
την αρχή
        if SB[j] > SB[j - 1]:           # Κριτήριο φθίνουσας ταξινόμησης (μεγαλύτερο προς
τα πάνω)
            # Αντιμετάθεση βαθμολογιών στη λίστα SB
            SB[j], SB[j - 1] = SB[j - 1], SB[j]
            # Παράλληλη αντιμετάθεση κωδικών στη λίστα KOD για διατήρηση της
αντιστοιχίας
            KOD[j], KOD[j - 1] = KOD[j - 1], KOD[j]

# Δ3.β: Δημιουργία και εγγραφή σε αρχείο κειμένου
f = open('lang.txt', 'w')  # Άνοιγμα (ή δημιουργία) του αρχείου για εγγραφή ('w')
for i in range(300):      # Επανάληψη για όλα τα στοιχεία των ταξινομημένων πλέον λιστών
    # Εγγραφή γραμμής: Κωδικός + κενό + Βαθμολογία (μετατροπή σε str) + αλλαγή γραμμής
(\n)
    f.write(KOD[i] + " " + str(SB[i]) + "\n")
f.close()                               # Κλείσιμο αρχείου και οριστικοποίηση εγγραφών
```